Perfect Consign — What It Takes to Be the One They Leave For

A strategy document for the owner. Written at HEAD `8c913c1`, after six rounds of hostile review and one build plan. This is not a feature list. It is an argument about what kind of product wins this category, why the thing you have already built is the right foundation for that product, and what has to be true before a competitor would rather buy you than compete with you.

The thesis in one paragraph

Every consignment and vendor-mall system on the market is an inventory list with a payout report bolted on. SimpleConsign, ConsignCloud, Ricochet, Liberty, Quail — all of them track items, ring sales, and tell the owner what each consignor is owed. None of them keeps books. You do. Perfect Consign is a double-entry ledger with a store wrapped around it: append-only journals, posting keys that make every money movement idempotent, schema-per-tenant isolation, control accounts for booth deposits and sales tax, sixty-four SQL proofs that run before every merge. That architecture is the moat, and it is a moat competitors cannot dig after the fact, because you cannot retrofit append-only onto a system whose customers have been editing history for ten years. The product that wins this category is the one that takes that correctness and makes it *felt* — by the consignor who finally trusts the number, by the owner who stops reconciling on Sundays, by the accountant who gets a trial balance instead of a spreadsheet, and by the acquirer whose diligence team finds that every claim in the README is proven by a test. The work ahead is not to add features until you match the others. It is to make correctness the reason people switch.

Part 1 — Why people actually switch

Nobody leaves their POS because a competitor has one more report. Switching is painful: re-tagging inventory, re-training staff, re-onboarding every consignor, and a month of the two systems disagreeing. People leave for one of four reasons, and the order matters.

They leave because **they were burned**. A statement was wrong and a consignor caught it. A payout went out twice. The sales-tax report did not match what was deposited and the state noticed. An employee voided sales and pocketed cash and the system had no record. Every store owner who has run a shop for more than three years has one of these stories, and the story is what they tell the next vendor's salesperson. This is where your ledger is not a technical nicety but the whole pitch: *in this system, a posted sale cannot be edited, a void is a second journal that references the first, and every cent owed to every consignor is a balance in a control account that reconciles to the general ledger every night.* No one else in the category can say that sentence truthfully.

They leave because **their consignors are unhappy**. The consignor relationship is the business. A consignor who does not trust the numbers takes their good inventory to the shop across town. The single most common complaint is opacity: "did my couch sell?", "why is my check smaller than I expected?", "what happened to the lamp?". The portal you just built is the beginning of the answer. The world-class version is a portal where the consignor sees the *same ledger the owner sees*, filtered to their rows, with every figure traceable to a sale, a markdown, a fee, or a payout — and where the number on the screen is, by construction, the number on the check.

They leave because **the day-to-day is slow**. Intake takes too long, the register freezes when the wifi hiccups, the end-of-day takes forty minutes, payout day takes a whole afternoon. These are the complaints that rarely trigger a switch on their own but that make a store receptive to the pitch when one of the first two happens. Offline tolerance, which you have started, and mobile intake, which you have not, live here.

And they leave because **the business outgrew the tool**. A second location. A mall operator who now runs three malls. A shop that started selling online and needs one inventory across channels. An owner who wants to sell the business and needs clean books for the buyer. The multi-tenant architecture and the real books mean you are already better positioned for this than anyone else, but the product does not yet *present* that way.

The strategy that follows is organised around those four reasons, in that order, because that is the order in which they produce switches.

Part 2 — Make correctness visible: the trust layer

The ledger is invisible today. It is the reason the numbers are right, but nothing in the product shows the user that it exists or lets them lean on it. The highest-leverage work in the whole roadmap is to surface it.

The consignor statement as a proof, not a report. Every monthly statement should be generated from the ledger, not from a query over sales, and every line should link to its journal. Opening balance, each sale with its commission and the rule that produced that commission, each markdown with the schedule step that triggered it, each fee, each payout with its check number or ACH trace, closing balance — and the closing balance *is* the control-account balance for that consignor as of the statement date, so the statement cannot disagree with the books. Put a small line at the bottom: "This statement reconciles to the store's general ledger. Statement hash: …". Nobody in the category does this. It costs you almost nothing because the data is already there. It is the thing an owner will show their accountant, and it is the thing a consignor will show their friend who consigns across town.

An owner-facing "books are closed and balanced" ritual. End of day should end with a screen that says, in plain language: cash in drawer matches posted sales plus float less payouts, to the cent, or it does not and here is the variance as a journal entry that someone has to sign. Sales tax collected today is now in the sales-tax-payable account. Consignor payables increased by this much. The register session is closed and cannot be reopened. Owners who have lived with systems where "close the day" means "print a report" will feel the difference immediately. Period close should be the same ritual at month end, and the existing `prove_*` discipline should be exposed as a user-visible "integrity check" that runs on demand and reports green.

Every number clickable to its source. The report screens should not be the end of a query; they should be the top of a tree. Sales by category drills to the sales, each sale to its journal, each journal to its lines and its register session and the cashier and the audit row. This is what "real books" means to a non-accountant: the ability to ask "where did this number come from?" and always get an answer. Competitors cannot build this because their numbers do not come from anywhere; they are sums over mutable rows.

The audit trail as a product feature. The admin portal's `admin_audit` table — append-only, guarded by event triggers, with outcome and target and details — is exactly the right shape. Bring it to the tenant side for every action that touches money or inventory or a person's record, and then *show it*: an activity feed per item ("intake by Dana on March 3, priced at 45.00, marked down to 36.00 by the 30-day schedule on April 2, sold by Marcus on April 9, consignor paid on May 1 by check 1042"), per consignor, per register session, per staff member. A store owner who suspects shrinkage should be able to answer the question in five minutes without calling you. This is also the feature that makes a buyer's diligence team relax, because it is what they will look for first.

Explain the money at the counter. When a cashier rings a 50.00 item for a consignor on a 40% tiered rule with a 10% promotion, the receipt and the sale screen should be able to show the breakdown on request: price, discount and why, tax and rate, commission and which rule, consignor's share. The commission-rule engine already resolves the rule; make it say which one it picked. Disputes at the counter die when the reason is on the screen.

None of this is a new subsystem. It is presentation over a ledger you already have. It should come before any new feature, because it is what turns "we have real books" from a line in your pitch into a thing the customer experiences every day.

Part 3 — The consignor as a first-class user

The portal exists now. Making it the best in the category is about three things: trust, convenience, and self-service that reduces the store's workload rather than adding to it.

Trust is Part 2 applied to one person. The consignor sees their items with photos, status and location, their sales with the full breakdown, their running balance as a ledger with every entry explained, their payout history with check numbers, their agreement with the actual commission rules and markdown schedule that apply to them, and the items about to expire with the date and what will happen. All of it read from role-scoped views, as the narrow database role built for the purpose, so that the guarantee "you can only see your own rows" is a property of the database and not of the application code. The hostile review's N34 is not just a security finding; it is the difference between a portal you can say that sentence about and one you cannot.

Convenience means meeting the consignor where they are. A magic link by email is right; add SMS as an option because half of them will not check email. A push notification when something sells, because that is the dopamine hit that makes a consignor bring in more. A weekly digest that says "three items sold, 84.50 earned, 212.00 balance, payout on the 1st". A one-tap "I'd like store credit instead of a check this month" that writes the preference and nothing else. A downloadable annual summary in January for the ones who need it for taxes.

Self-service that reduces work is the part to be careful about. The build plan was right to exclude self-service intake and request-a-payout buttons for now, because they generate staff work and disputes. But there is a middle ground that competitors miss. Let the consignor *pre-register* items from home — photos, description, their asking price — into a queue that the store accepts, re-prices, and tags on arrival. The store still controls everything; the consignor has done the data entry. Let the consignor *respond to an offer*: when an item has sat for sixty days, the system can email "we propose marking this down to 30.00 or returning it; reply by Friday", and the markdown queue you just built becomes a conversation instead of a surprise. Let the consignor *schedule a pickup* of expired items from a list of slots the store publishes. Each of these moves work from the counter to the consignor's couch and each one produces a timestamped record that ends the "nobody told me" dispute.

Donation and expiry done properly. When an item expires to donation, generate the charity receipt with the item description and the consignor's name, mail it, and record it. Nobody does this well. It is cheap, it is noticed, and it is one more piece of paper the consignor gets from you and not from the shop across town.

Part 4 — The counter: offline, fast, and mobile

You have said offline matters most, and the review has just found that the offline path, while idempotent, trusts the browser about commission and tax. Fixing that is the first task. Beyond the fix, the world-class register has three properties.

It is never the reason a sale is lost. The current design — a `localStorage` outbox that replays on reconnect — is a good first step, but it only engages when the pay request fails. The next step is a register that keeps working through an outage: a service worker that caches the register shell and the current price list, so scanning, pricing and tendering continue with a visible "offline — 3 sales waiting" badge, and the outbox drains itself when the network returns. The ledger side of this is already right: posting keys make replay safe, and the sync function returns the same journal on a duplicate. What remains is the client, and the rule that the server, not the client, decides commission and tax at sync time. An outage should produce a slightly delayed journal, never a different one.

It is fast in the hands of a part-timer on a Saturday. The register should be keyboard-first and scanner-first, with the common path — scan, scan, cash, done — needing no mouse. Layaway, holds, store credit, gift certificates and split tenders should each be one key. The pay modal should quote cash-due with nickel rounding where the store has turned it on, which you already do. The lookup for an item with a lost tag should be fast enough to use with a queue watching. Measure it: time from first scan to drawer open, on the store's actual hardware, with the store's actual catalogue size. Publish the number.

Intake happens where the item is. This is the biggest day-to-day gap against ConsignCloud and SimpleConsign. A staffer should be able to walk an estate or a consignor's car with a phone: photograph, describe, pick a category, accept the price the rules suggest or override it, print or queue the tag, next. The same offline discipline applies — intake records get client-generated keys and sync when they can. Photos are the heavy part; take them at a sensible resolution, upload in the background, and let the item exist before the photo arrives. The pricing suggestion is where the ledger quietly helps: "items like this from this consignor sold in a median of 23 days at 85% of ask" is a query over your own sales history, and no competitor has sales history they can trust enough to say that.

Tags and labels are a product, not an afterthought. Thermal label support for the two or three printers stores actually own, a QR code that opens the item in the portal, a sticker layout the owner can adjust once and forget. The re-tag list that the markdown queue produces should be a one-click print of only the labels that changed.

Part 5 — The money: payouts, cards, taxes, and the accountant

You have deferred ACH and integrated card processing, and that is a reasonable sequencing choice. But the strategy has to say where they fit, because they are both the places competitors beat you in demos and the places where the next revenue line lives.

Payouts. Checks are fine for the sixty-year-old consignor and useless for the thirty-year-old. ACH through a provider that handles the compliance (Stripe Connect, Dwolla, Modern Treasury) turns payout day from an afternoon with a check printer into a button. The ledger makes this unusually safe for you: a payout is a journal that debits the consignor's payable and credits a clearing account; the ACH trace id goes on the journal; a returned payment is a reversing journal. Statement, balance and payout history stay consistent without any special handling. When you build it, build the store-credit path in the same motion — a consignor choosing credit over cash is a transfer between two of your own liability accounts and it should be a one-line journal, not a workflow.

Card processing. You record tender; competitors process it. The gap is not the swipe — any terminal can do that — it is reconciliation. A store on SimpleConsign gets a daily settlement that matches their sales because the same company owns

both sides. The way to close this without becoming a payments company is a semi-integrated terminal (Stripe Terminal, Square Terminal, or a PAX device through a gateway) where the register initiates the charge, the terminal takes the card, and the result comes back with a transaction id that goes on the journal. Then the settlement file from the processor reconciles against your card-clearing account automatically, and the owner never types a batch total again. That reconciliation is a feature you can build better than anyone because you have a clearing account and they do not.

Sales tax. You already treat sales tax as a liability account with a filing report, which puts you ahead. The world-class version handles the edge cases stores actually hit: tax-exempt consignors and buyers with the certificate on file, multiple jurisdictions for a mall near a county line, tax-inclusive pricing for the stores that do it, and a filing export that matches the state's form line for line. Where the offline path collects tax at a stale rate, post what was collected and show the variance; never silently correct it.

1099s. Any consignor paid over the federal threshold needs a 1099-NEC in January. Collecting the W-9 at agreement time, tracking the year-to-date total as a query over the payable account, generating the forms, and offering e-file through a provider is two weeks of work and it is the kind of thing that makes a serious store choose you over a hobbyist tool. The ledger makes the total trustworthy; most competitors' 1099 numbers are sums over a payout table that has been edited.

The accountant. QuickBooks Desktop export for your one tenant is in the build plan and should be done. The world-class position is broader: a general-ledger export in a standard shape (journal date, account, debit, credit, memo, reference) that any accountant can import into anything, plus native sync for QuickBooks Online and Xero when the demand is there. Position it honestly: "we are the system of record; your accounting package receives a summarised daily journal from us and the two will always agree". No competitor can say that, because none of them has a journal to summarise.

Part 6 — The owner: merchandising intelligence from books you can trust

The previous competitive analysis warned against building the third-best merchandising dashboard before the portal existed. The portal exists. Now the warning changes: do not build *their* dashboard; build the one only you can.

Every competitor's analytics are sums over mutable tables, and owners learn not to trust them. Yours can be derived from an append-only ledger with full history, which means you can answer questions they cannot. Sell-through by category, brand, consignor and price band is table stakes; do it. Aging buckets with the markdown schedule overlaid, so the owner sees "these 140 items hit the 40% step next Tuesday and here is the dollar value" — that is yours alone. Consignor performance as a real contribution-margin number: revenue from their items, less commission paid out, less the cost of the floor space and labour their items consumed, so the owner can see which consignors to court and which to decline politely. Days-to-sell and realised-versus-asked by category, feeding back into the intake price suggestion. For the mall side, revenue per square foot by booth, rent collected versus rent due, deposit liabilities, and a waiting-list conversion rate.

Make the "what should I stop taking" question answerable. Make the "which day of the week should I staff up" question answerable from register sessions. Make the "what did I actually make last month" question answerable from the income statement, not from a sales total with the owner's mental arithmetic for commission and rent.

Deliver these as a handful of opinionated screens, not a report builder. Owners do not want to build reports; they want to be told what matters. Each screen should drill to the journals (Part 2), and each should be exportable as CSV with the formula-injection guard you already have.

Part 7 — Growing with the business: multi-location, marketplaces, and the mall

Multi-location. The schema-per-tenant architecture is a strength; the product should present a tenant as a *business* that may have several locations rather than forcing a second tenant for a second store. Inventory that can move between locations with a transfer journal, a consignor who consigns at one location and is paid from the same balance regardless of where the item sold, consolidated and per-location books, staff who can be scheduled at either. This is a reporting and data-model change more than a new system, and it is the feature that lets you sell to the operator with three stores, who is also the operator most likely to be acquired by a chain that then needs twenty.

Online channel. You have said online selling is not a priority now, and it should not displace the counter and the portal. But the architecture decision should be made now, because it determines whether it is a month or a year later. The decision is: inventory has one source of truth, which is your ledger, and every channel is a *listing* of an item that may sell in one place only. When an item sells in the store, the listing is withdrawn; when it sells online, the store sale is a reservation that posts when the order is paid. Shopify first, because the stores with a following already have one and the integration is well-trodden, then eBay and Poshmark for the categories where they dominate. Build the listing abstraction (item → listings → channel) into the schema before you need it, with the sale journal recording which channel earned it, and the rest becomes

connectors.

The vendor mall. Booth leases are in the build plan and partly built. The world-class mall product adds autopay for rent (which is the ACH rail again), a floor map that shows occupancy, arrears and lease expiries at a glance (you have the map), a waiting list that converts automatically when a booth opens, vendor self-service for rent payment and sales reporting through the same portal as consignors, and the deposit liability handled as the control account it already is. Mall operators are a smaller market than consignment shops, but they are stickier, they pay more, and the booth-level trial balance is something no one else can show them.

An API and webhooks. Not for its own sake, but because every integration above becomes cheaper with one, and because a buyer values a platform more than an application. Read-only first — items, sales, consignor balances, journals — with per-tenant keys and the same role scoping as the portal. Webhooks for "item sold", "payout posted", "item expiring". Document it publicly. The accountant who wants to pull the trial balance nightly, the store with a developer friend, the integration partner who wants to list inventory — all of them become possible without you writing the code.

Part 8 — Migration: lowering the cost of leaving the other system

The reason people stay with a bad system is that leaving is worse. The product that wins is the one that makes leaving easy. This is unglamorous and it is possibly the highest-return engineering on this list.

Build an importer for each major competitor's export: items with photos, consignors with balances and agreements, sales history where available, open layaways, store credit balances. Treat the imported opening balances as what they are — an opening journal that credits each consignor's payable account with the balance the old system claimed — so that from day one the new books are real books and the old system's errors are isolated in a single, visible, dated entry that the owner can reconcile and adjust. Offer a parallel-run period where both systems ring sales and you produce a nightly reconciliation showing where they differ. The store that sees your books disagree with their old system by 212.00 on day three, and then finds that your number was the right one, is a store that never leaves.

Then do the reverse. Publish a complete, documented export of everything — items, consignors, journals, audit trail — in open formats, and say so loudly: *your data is yours and you can take it with you in one click*. Stores have been locked in by vendors who make export deliberately painful. Being the vendor who does not do that is a sales argument, and it is also the honest position for a product whose whole pitch is trustworthy records.

Part 9 — Security and operations as a sales asset

Six rounds of hostile review have produced a security posture better than anything in the category: per-tenant database roles, `SECURITY DEFINER` boundaries, append-only enforcement in the schema, sealed secrets, stage-classified mail failures, lockouts on every credential path. The findings that remain — Host-header reset poisoning, the admin session model, the consignor portal running as the wrong role, a red test suite at HEAD — are fixable in a week and the review says how. Once they are fixed, *say so*. Publish a security page that describes the architecture in plain language. Get an independent penetration test and publish the summary. Pursue SOC 2 Type 1 when the first multi-location customer asks, because they will. A store owner's IT-literate nephew will read that page; a mall operator's insurer will ask for it; an acquirer's diligence team will start there.

Operationally, the product should be as hard to run badly as it is to misuse. Automated backups verified by actually restoring them, with the restore tested in CI the way `wipe-backup` is now. A dead-letter alert so a failed job is a page, not a silence. A status page. A soft-delete window on tenant wipe so a phished admin cannot destroy a business in a minute. Secret scanning in CI. Response headers. An upgrade path that is documented and boring, because the forward-only migration machinery you have built is already most of it.

Support is part of the product. A store owner at 9 a.m. on a Saturday with a register that will not open does not care about your architecture. Publish a phone number, answer it, and make the register's error messages say what to do rather than what went wrong.

Part 10 — Business model and packaging

Price on value, not on seats. A flat monthly fee per location with consignor count and transaction volume as the tiers, so a growing store pays more and a tiny one can start. Do not charge for the portal; it is what sells the product to the consignor, and the consignor is who sells the product to the next store. Do charge for the money rails when you build them — a small

per-payout fee on ACH and a share of interchange on integrated cards are the revenue lines that turn a software business into a payments business, and payments businesses are what acquirers pay multiples for.

Offer a free, time-boxed parallel run with the migration tooling from Part 8. Offer an annual plan with the first month free. Offer a referral credit to a consignor whose recommendation brings a store, because consignors talk to each other across stores.

Keep the product single-codebase and single-deployment. Resist the custom build for the large customer. The multi-tenant architecture is an asset only if every tenant runs the same code.

Part 11 — What an acquirer is actually buying

A competitor pays millions for one of four things: customers they cannot win, revenue they cannot replicate, technology they cannot build, or a team they cannot hire. The strategy above is aimed at all four, but the technology case is the one you can make today and the one worth understanding precisely.

An acquirer's technical diligence team will ask: can we trust the books in this system? Is tenant data isolated by something stronger than a `WHERE` clause? Can we upgrade it without downtime? Is the test suite real? Is there a security debt we will have to disclose? Can we migrate our customers *onto* it? For the first time in this category, the answers can be yes, yes, yes, yes, no, and yes — but only if the work in the hostile review is done and kept done, because the first thing they will do is run `composer check`, and the second is grep for `Host`.

Beyond technology, the acquisition narrative is this. The incumbents have large customer bases on architectures they cannot fix: mutable sales tables, shared-schema tenancy, payouts computed by summing rows. They know it. Their customers do not, until they are burned. A product that keeps real books, isolates tenants in the database, migrates a competitor's customer in a weekend with an opening journal that reconciles, and shows the consignor a statement that proves itself, is a product that threatens their base one burned customer at a time. The rational response is to buy it and move their customers onto it. The ledger-first architecture is what makes that migration possible, and it is what they cannot build without starting over.

So the order of work, in the language of this document: fix what the review found and keep the suite green, because it is the first page of diligence. Surface the ledger so correctness is felt, because it is the pitch. Finish the consignor portal on the right role, because it is the demo. Harden offline so an outage never produces a wrong journal, because that is the promise. Build the importers, because they are the growth engine. Then the money rails, then multi-location, then the online channel, each on the schema you have.

The last thing worth saying is about discipline. Six rounds of review have shown a pattern: the architecture is right and the author closes findings properly, but features ship with tests red, coverage below the floor, and the occasional shortcut — a client-trusted commission, a role built and not used. In a company trying to be acquired, those are not engineering details; they are the difference between a diligence report that says "exceptional" and one that says "promising but inconsistent". The product is already the most correct one in its category. The strategy is to be the most *trusted* one, and trust is earned by never shipping red.
